MONAI の使い方

CT・MR のシリーズから、臓器を自動で塗り分けます。計算は外部の GPU（Google Colab など）で行い、結果は ROI マネージャで確かめて、DICOM SEG として保存します。

外部の GPU　DICOM SEG　研究用

この冊子の内容

01 プラグインを入れる
設定 > プラグイン

02 GPU の設定
Google Colab ／ この PC ／ 自分の Jupyter

03 実行と同意
何が外へ送られるか

04 結果を確かめる
下見・ROI マネージャ

05 SEG の意味と保存

06 モデルの選び方・困ったとき・既知の制限

研究用です。診断には使わないでください。データ: TCIA HCC-TACE-Seg（Moawad et al., 2021）CC BY 4.0　doi.org/10.7937/TCIA.5FNA-0924

全体の流れ

1

設定

外部の計算機（GPU）を 1 つ用意します。最初の 1 回だけです。

2

選ぶ

2D ビューアでシリーズを開き、解析 ＞ AI から、モデルを選んで「実行」。

3

同意

送るデータと実行するコードの全文を見て、承諾します。送るたびに必ず出ます。

4

確かめる

下見の画像とラベルを見て、ROI マネージャで確かめます。

5

保存

保存するラベルを選び、「SEG で保存」。元の画像は変わりません。

必要なもの

GRAPHY-Next 0.4.0 以降（デスクトップ版）
この冊子の画面は 0.5.1 です

MONAI Bundle プラグイン 0.2.0
公式プラグイン。署名つきで配布

Google アカウント（Colab を使う場合）
無料版にも利用枠の上限があります

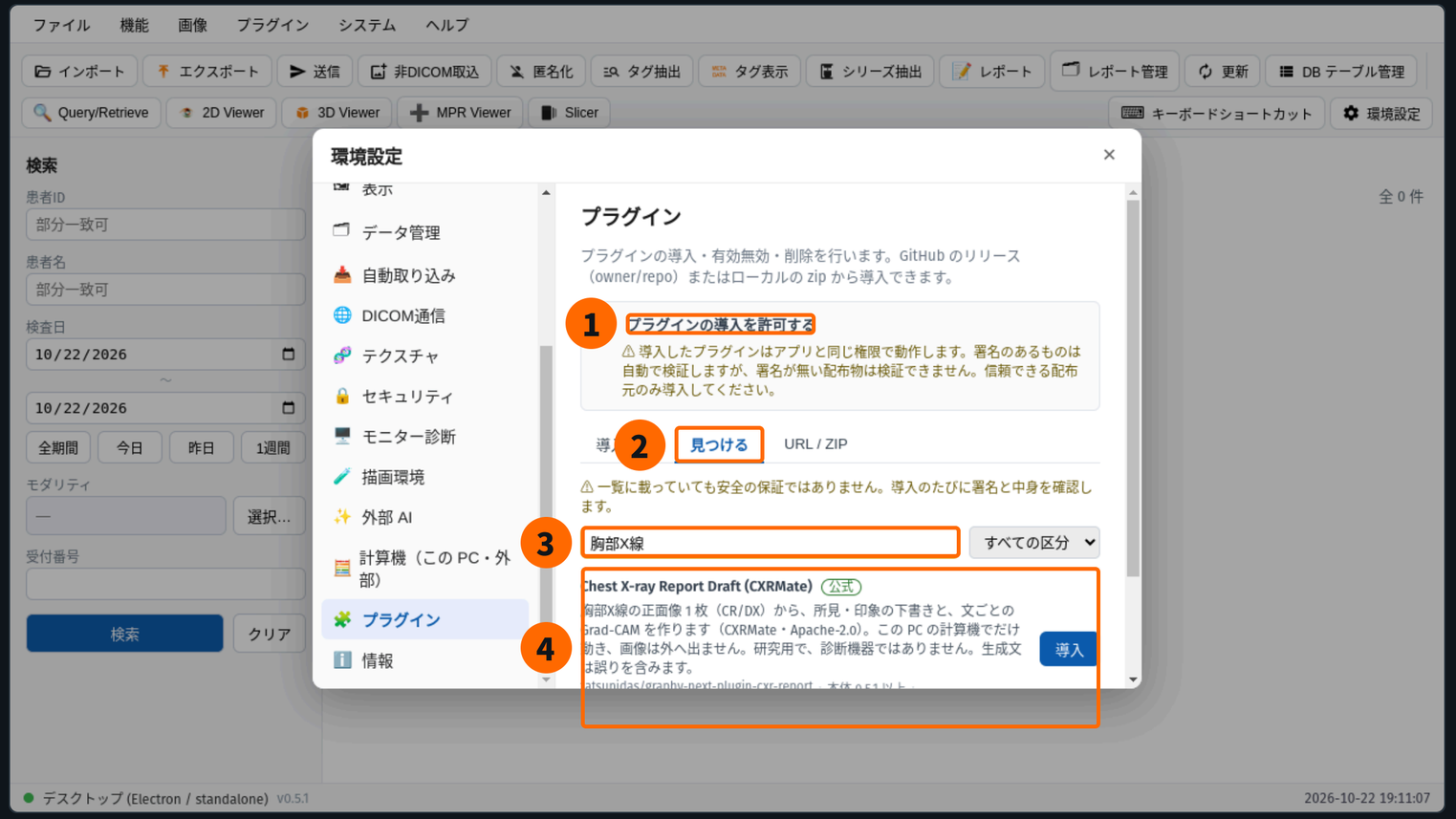
ご注意

研究用です。診断には使わないでください。
結果は必ず画像で確かめてください

送るのは、本体が**匿名化した**シリーズだけです
プラグインは画素を外へ送る口を持ちません

モデルは MONAI Model Zoo の公開モデルです
ライセンスはモデルごとに違います（非商用のものがあります）

公式の索引から、プラグインを導入する



1. 「プラグインの導入を許可する」を、**ご自身で**入れます（既定は許可しません）
2. 「見つける」に、公式の索引の一覧が出ます
3. 「MONAI」で絞り込みます
4. 「導入」。導入のたびに、署名と中身を確かめます。信頼欄が **verified** なら公式の署名です

> この図は別の公式プラグインの一覧です。MONAI も同じ手順で入ります。

ネットワークの無い環境では「zip から導入」も使えます（注意は p.13）。

計算する場所は 3 通り。まずは Google Colab がおすすめです

Google Colab

おすすめ。自分のパソコンに GPU や Python の用意が要りません。

Google でログイン → GPU を選んで追加

計算は**ご自身の Colab の利用枠**で行われます

パッケージは Colab 側で自動で入ります

この PC

この PC の Python と GPU を使います。

Python と **PyTorch・MONAI など**が要ります

足りないものは、実行の前に表と**導入コマンド**で案内します

環境は書き換えません（入れるのはご自身）

自分の Jupyter Server

自分で立てた Jupyter Server（社内の GPU サーバーなど）を使います。

設定の「計算機を追加」から登録します

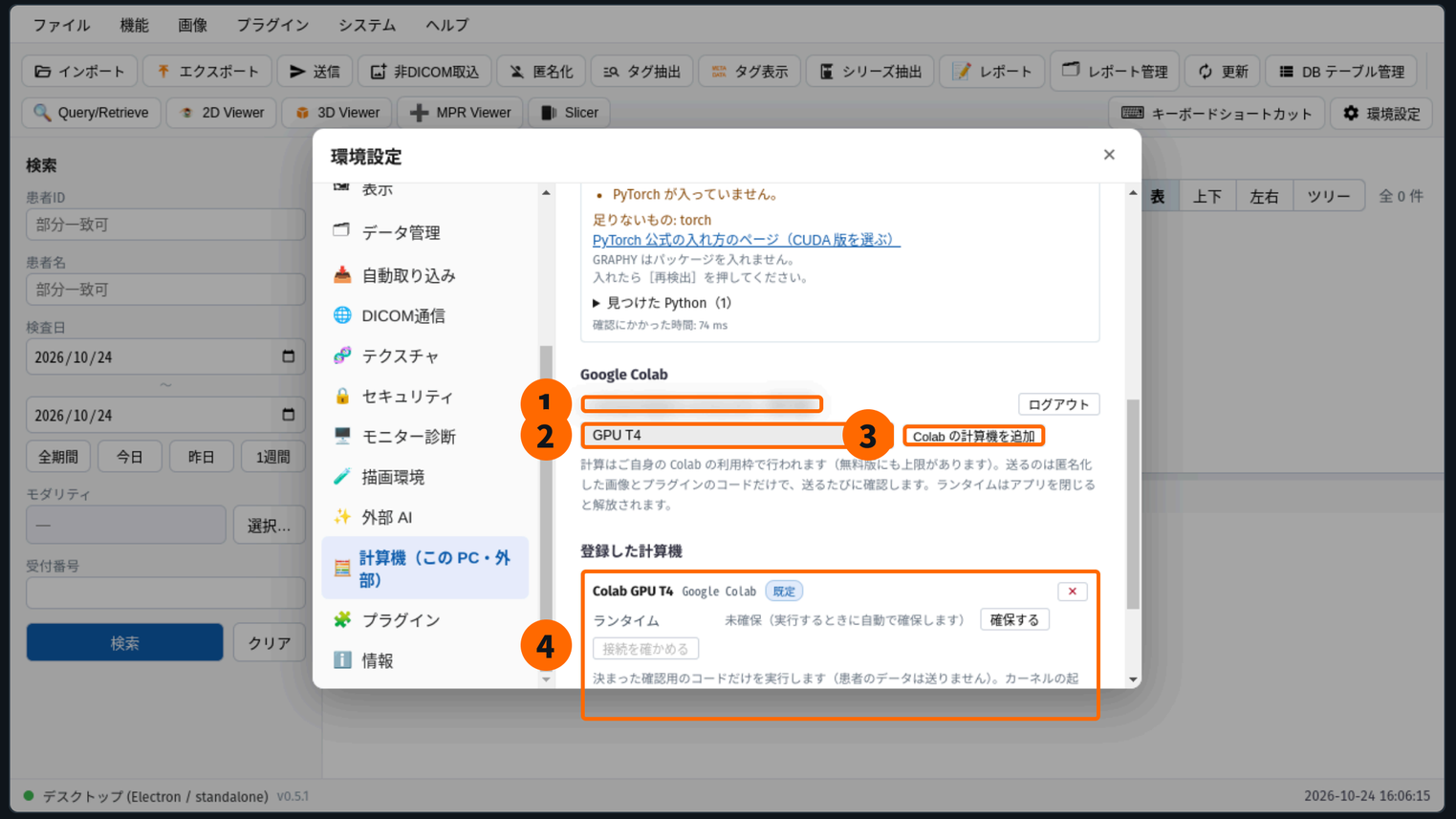
必要なパッケージは「この PC」と同じです。足りないものは同じ枠で案内します

この冊子では画面を省略します

> 計算機を 1 つも登録していないときは、最初の「実行」で、Colab の GPU T4 が自動で登録されます（Colab にログインしている場合）。

どの計算機でも、実行のたびに「何を・どこへ送るか」の同意画面が出ます。承諾するまで何も送られません。

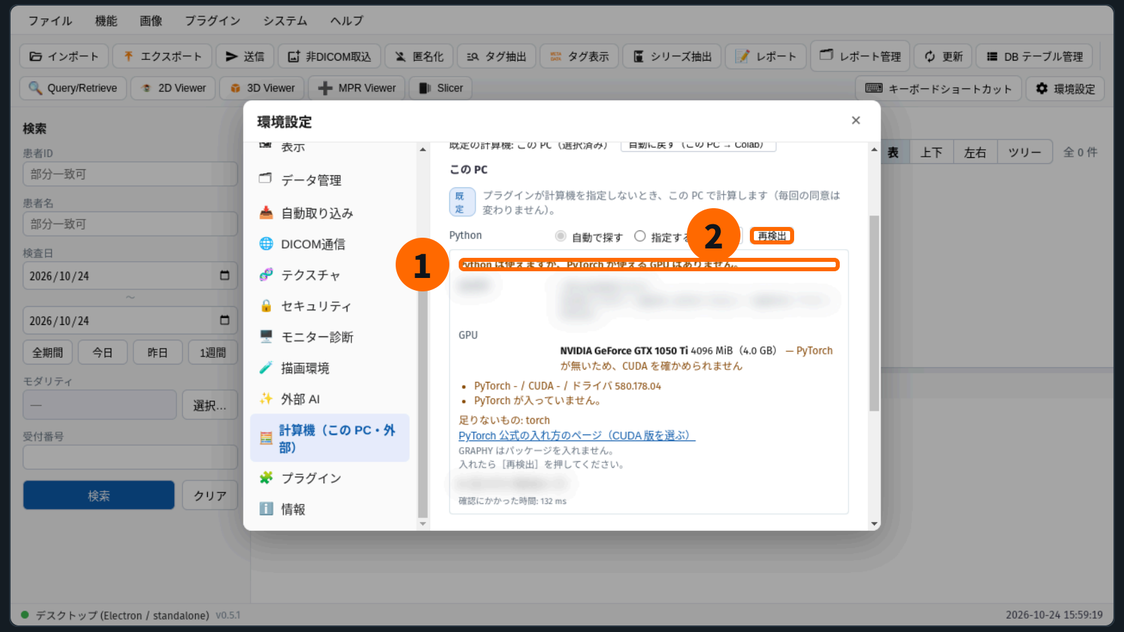
Colab にログインして、GPU の計算機を追加する



1. 「Google でログイン」→ ブラウザが開くので、Colab を使うアカウントでログイン。アカウント名がここに出ます（図ではぼかしています）
2. GPU の種類を選びます。まずは **GPU T4**。プランで使えないものには「このプランでは使えません」と出ます
3. 「Colab の計算機を追加」
4. 「登録した計算機」に出ます。**既定** の印がついた計算機が、プラグインの実行に使われます

> Colab の計算機の実体（ランタイム）は、**実行するときに自動で確保**します（数秒〜数十秒）。

足りないものは、実行の前に表と導入コマンドで案内します



1. 設定では、この PC の Python と GPU を見分けた結果が出ます
2. 環境を変えたら「再検出」



1. 最初の「実行」で、**画像を送らない**確認が先に走ります。足りないものを表にします
2. 使われる Python を指した `pip install` を出します（コピーして実行）
3. 入れたら「再確認」。**アプリの再起動は要りません**

GRAPHY はパッケージを自動では入れません。GPU（CUDA）で動かすときは、PyTorch だけ公式サイトの案内の版を入れてください。図は、何も入っていない Python の例です。

研究用です。診断には使わないでください。Python の場所は隠しています。

シリーズを開き、モデルを選んで「実行」



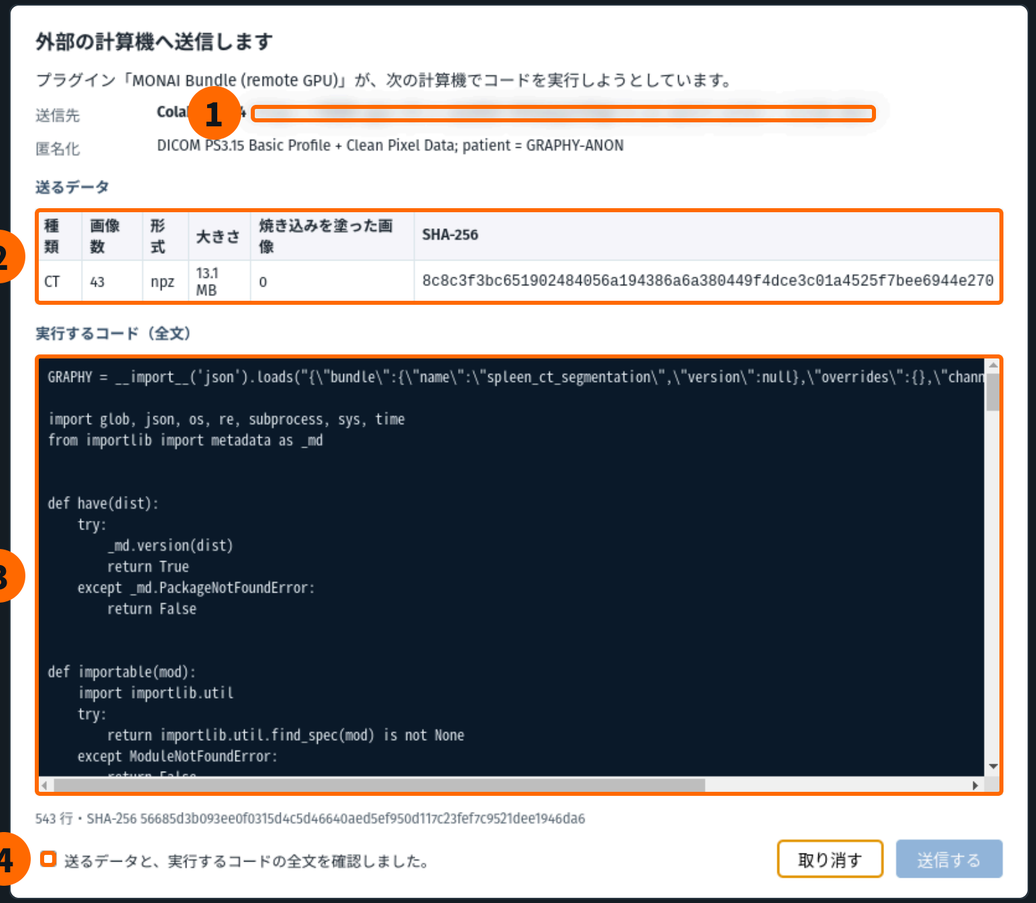
1. モデルを選びます。このシリーズに使えるものだけが選べます。まずは軽い **脾臓（spleen_ct_segmentation）** が試しやすいです
2. 版（既定は最新）
3. 「実行」。次の同意画面で、送る内容を確かめます

> 計算機の上で、モデルの説明（`metadata.json`）から**このシリーズに使えるか**を判定します。合わなければ、推論せずに止まり、理由が出ます。

⚠ モデルの説明は作者の自己申告です。止めるのは、明らかに合わないときだけです。

03 / 同意画面（実行のたびに出ます）

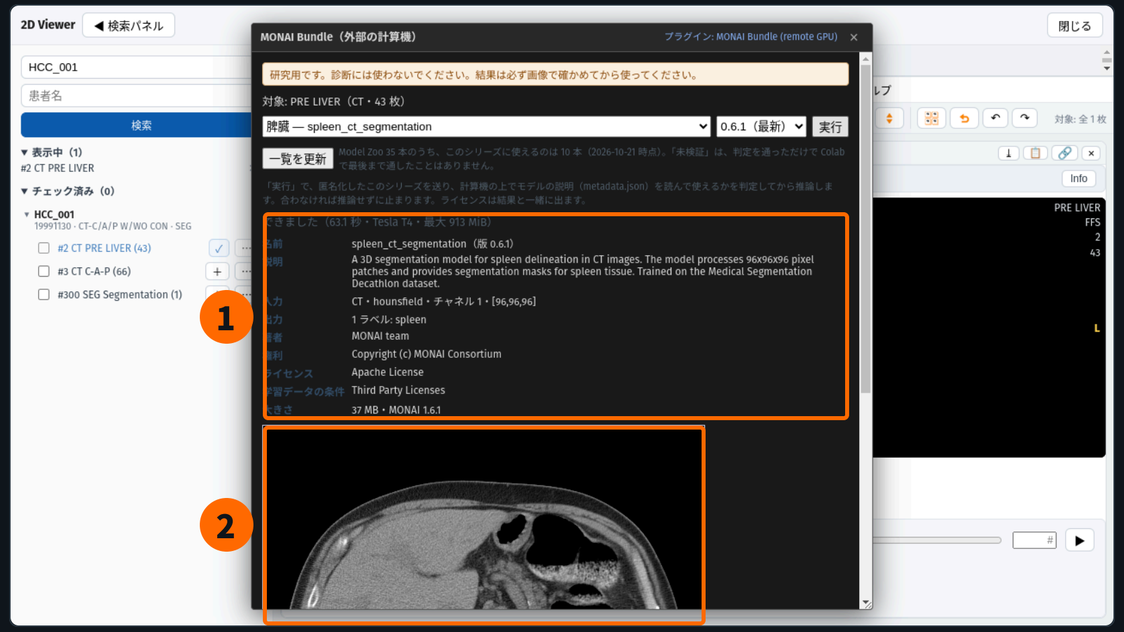
何を・どこへ送るかを見て、承諾してから送ります

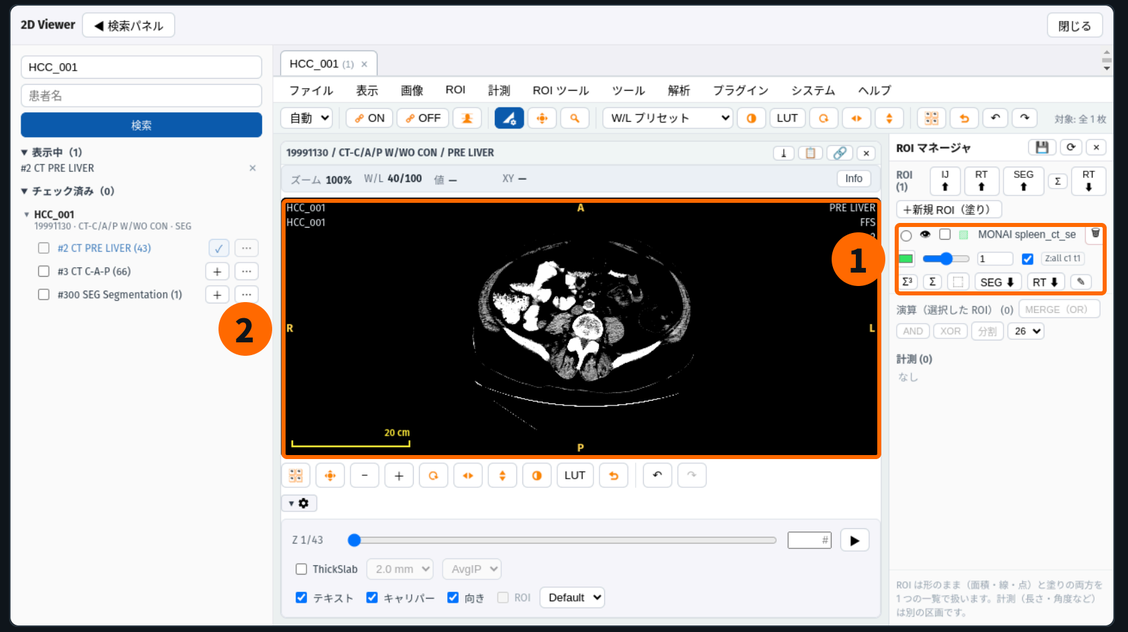


1. **送り先**。計算機の名前と URL です（この図では URL をぼかしています）
2. **送るデータ**。枚数・形式・大きさと、**焼き込み文字を塗った枚数**。患者は匿名（GRAPHY-ANON）に置き換わります
3. **実行するコードの全文**。計算機の上で動くプログラムを、すべて読めます
4. 確認したらチェックを入れて「送信」。やめるときは「キャンセル」で、何も送られません

> 最初の実行では、画像を送らない確認の同意が**先に 1 回**出ます（Colab では何も確かめずに通ります）。

下見の画像とラベルを見て、ROI マネージャで確かめる





1. 終わると、時間・GPU・モデルの説明が出ます。この例は **63 秒**（Tesla T4）
2. **下見**。塗られた臓器が見えます。ラベル横の数は**ボクセル数**
3. 窓を下へ動かすと、保存するラベルのチェックと「SEG で保存」があります（p.11）

1. 結果は ROI マネージャにも**臓器名つきで**読み込まれます（ROI ツール ＞ ROI マネージャ）
2. **スライスを動かして、塗りが臓器に合っているかを必ず確かめてください**

SEG は「どの画素が、どの臓器か」の記録です

元の CT シリーズ

画像そのもの（患者・検査・スライスの位置）
変更されません

＋

SEG（新しいシリーズ）

臓器ごとの塗り（セグメント）。1 臓器 ＝ 1 セグメント。名前（例: spleen）と色がつきます
各スライスの**位置・向き**（患者座標・mm）を持ち、元の画像の格子に合います
元のシリーズへの参照。患者・検査の情報は元のシリーズから引き継ぎます
画像は含みません。塗りの 0/1 だけです（BINARY 形式）

ラベル番号と臓器名の対応は、モデルごとに違います。保存された SEG には、臓器名がセグメント名として入ります。

前景が無い（何も塗られなかった）ラベルは、保存されません。保存の確認画面に、セグメント数が出ます。

SEG は DICOM の標準の形式です。元のシリーズと一緒に開けるビューアで、塗りを重ねて見られます。

結果は**研究用**です。SEG に保存しても、診断に使える結果になるわけではありません。

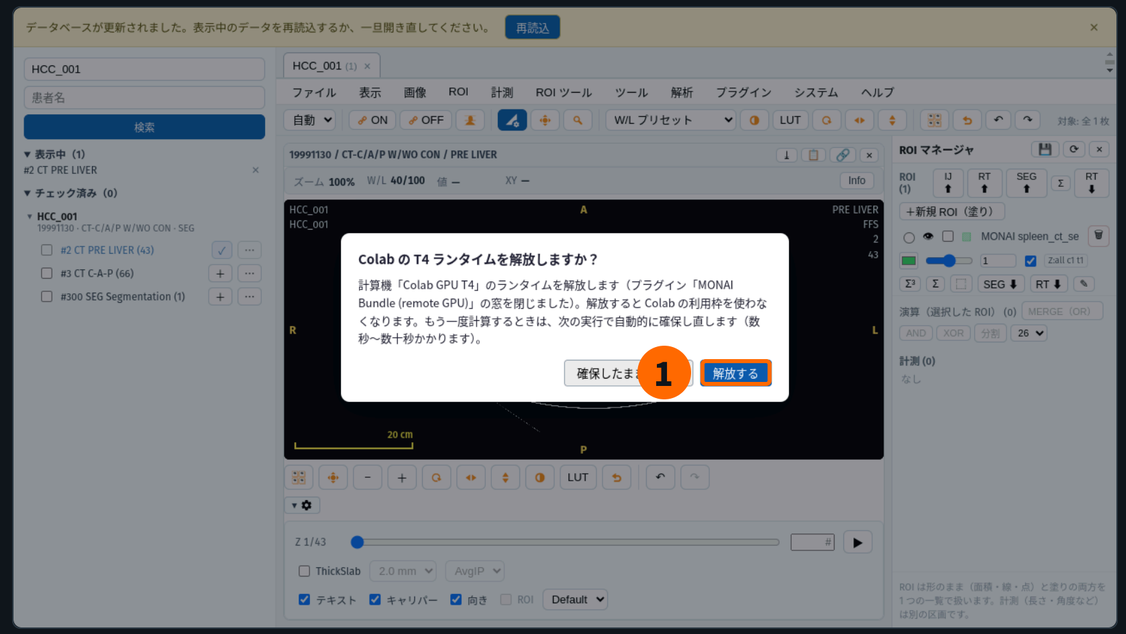
SEG の中の「アルゴリズムの種類」は、すべて `MANUAL` と書かれます（本体の既定）。AI が作ったことは、シリーズ説明の `[Plugin]` で見分けてください。

「SEG で保存」で、新しいシリーズとして入れる



1 確認画面で、シリーズ説明（`[Plugin] MONAI …`）・**セグメント数**・保存先を確かめて「保存する」。保存先は**この PC の保管庫**です

保存すると「SEG を保存しました」と出て、一覧に新しいシリーズが増えます（再読込で更新）。元の画像は変わりません。



1 プラグインの窓を閉じると、Colab のランタイムを**解放するか**聞かれます。続けて使わないなら「解放する」（Colab の利用枠を使わなくなります）

解放しても、次の実行で自動で確保し直します（数秒〜数十秒）。

T4 で最後まで通したのは 8 本。ほかは「未検証」

モデル	対象	ラベル	GPU の最大（T4）
spleen_ct_segmentation	CT	脾臓	0.9 GB
wholeBody_ct_segmentation	CT	全身 104 臓器	8.4 GB
swin_unetr_btcv_segmentation	CT	腹部 13 臓器	5.4 GB
multi_organ_segmentation	CT	腹部 7 臓器	5.8 GB
pancreas_ct_dints_segmentation	CT	膵臓・膵腫瘍	3.4 GB
renalStructures_UNEST_segmentation	造影 CT	腎臓の構造	5.9 GB
prostate_mri_anatomy	T2 MR	前立腺	4.0 GB
wholeBrainSeg_Large_UNEST_segmentation	T1 MR（MNI 空間）	脳 133 領域	11.5 GB

「未検証」は、判定を通っただけで、Colab で最後まで通したことがない、という意味です。試せますが、動く保証はありません。

一覧に無い Bundle も「その他」から名前で指定できます（Model Zoo の名前、または Hugging Face の 組織/名前）。

設定値・複数チャネル（脳腫瘍など）・ラベル番号と点（vista3d など）を渡せるモデルもあります。画面に入力欄が出ます。

ライセンスはモデルごとに違います（学習データの条件で、非商用に限られるものがあります）。結果の画面で確かめてください。

うまくいかないときと、いまわかっている限界

うまくいかないとき

出たこと	対処
足りないものの表が出た	表の `pip install` をターミナルで実行 → 「再確認」
Bundle が追加のパッケージを求めた	枠に出た名前を、同じ Python に入れて、もう一度「実行」
Jupyter などを後から入れたのに「local-unavailable」	GRAPHY を再起動（次の版で直す予定です）
Colab で GPU の空きが無い／利用枠を使い切った	時間をおく、別の種類を選ぶ
GPU のメモリ不足（CUDA out of memory）	軽いモデルにする。設定値で解像度を下げられるモデルもあります
署名つきで入れたプラグインを zip で入れ直せない	本体の次の版から、zip と同名の `.minisig` も一緒に選びます。0.5.1 では、いったん削除してから入れます

いまわかっている限界（実機の検証）

脾臓（10 例・独立データ）: Dice の平均 0.858（最小 0.536、最大 0.949）。**10 例すべてで、予測の体積が基準より小さい**傾向
1 例は脾臓の一部しか拾えませんでした。原因は未確認

全身 104 臓器（wholeBody）: 既定の設定では、T4 で**10 例すべて失敗**。確認できた原因は、メモリ不足の 1 例だけです
低解像度の設定では 7 例が完走。結果はばらつきます

向き・体積の検証は、**一部のモデルだけ**です。ほかのモデルは、必ず画像で確かめてください